Internship Report

Course Code: SWE–420

Course Title: Internship

Submitted To:

Department of Software Engineering
Institute of Information and Communication Technology (IICT)
Shahjalal University of Science and Technology, Sylhet

Submitted By:

Md. Rakibul Kabir
Registration No.: 2020831051
Session: 2020–2021
Department of Software Engineering, IICT, SUST

Performed At:

FlowGenX AI

Software Development
Headquarters: San Francisco, California, USA

Internship Period: 10 November 2025 – 02 May 2026
Date of Submission: 03 May 2026

Letter of Transmittal

May 4, 2026

Dr. Mohammad Reza Selim
Director
Institute of Information and Communication Technology (IICT)
Shahjalal University of Science and Technology, Sylhet

Subject: Submission of Internship Report for SWE–420.

Dear Sir,

With profound respect and great pleasure, I am submitting my internship report for the course **SWE–420**, undertaken in my eighth semester as a partial fulfilment of the Bachelor of Science in Software Engineering program. I successfully completed a full-time internship at **FlowGenX AI** from **10th November 2025** to **02nd May 2026**, working remotely from Bangladesh in the position of *Full Stack Software Engineer – Generative AI (Intern)*.

This report reflects the technical knowledge, skills, and industry exposure gained during my internship under the supervision of Mr. Sumon Saha (CEO & Founder), Mr. Sanjib Sen (Team Lead), and Mr. Abrar Bin Rofique (Software Engineer – Gen AI). I contributed to FlowGenX AI's `runtime_connectivity` project.

I have prepared this report with sincere effort and care. I would be honoured by your kind review and acceptance.

Yours sincerely,

Md. Rakibul Kabir
Registration No.: 2020831051
Session: 2020–2021
Department of Software Engineering
Institute of Information and Communication Technology
Shahjalal University of Science and Technology, Sylhet

Letter of Endorsement

To Whom It May Concern

Subject: Endorsement of Internship Report.

This letter is to certify that all the information presented in this report is authentic and reflects the genuine work undertaken by **Md. Rakibul Kabir** (Registration No. 2020831051), a final-year student of the Department of Software Engineering at the Institute of Information and Communication Technology, Shahjalal University of Science and Technology, during his internship at **FlowGenX AI** from **10$^{\text{th}}$ November 2025** to **02$^{\text{nd}}$ May 2026**.

During this period, he served as a *Full Stack Software Engineer – Generative AI (Intern)* and contributed to the `runtime_connectivity` project, where he designed, implemented, tested, and documented a significant number of new connectors that have been integrated into the FlowGenX AI platform. The contents of this report have been reviewed prior to submission and are not considered confidential.

I am happy to ensure the validity of the report and wish him every success in his future career as a software engineer.

Mr. Sumon Saha
Founder & CEO of FlowGenX AI

Mr. Balaji Sundara
Vice President, GTM & Growth
FlowGenX AI

Mr. Sanjib Sen
Senior Software Engineer
(Team Lead)
FlowGenX AI

Mr. Abrar Bin Rofique
Software Engineer (Gen AI)
(Internship Supervisor)
FlowGenX AI

FlowGenX AI
10456 Stokes Avenue, Cupertino, CA 95014
California, USA
flowgenx.ai
May 3rd, 2026

Certificate of Completion

To Whom It May Concern,

This letter certifies that **Rakibul Kabir** successfully completed his internship/engagement with **FlowGenX AI** from **November 10, 2025** to **May 2, 2026**.

During his time with FlowGenX AI, Rakibul contributed as part of the engineering team and supported software development activities related to the company's AI-driven integration and orchestration platform. His work involved collaborating with team members, assisting with development tasks, and gaining practical experience in a modern software product environment.

Rakibul demonstrated professionalism, sincerity, adaptability, and a strong willingness to learn throughout his engagement. We appreciate his contributions to FlowGenX AI and wish him continued success in his academic and professional career.

Sincerely,

Sumon Saha
Founder & CEO
FlowGenX AI, California, USA

ssaha@flowgenx.ai

+1 512 587 4398

Acknowledgement

First and foremost, I would like to express my heartfelt gratitude to the **Department of Software Engineering** and the **Institute of Information and Communication Technology (IICT)** at Shahjalal University of Science and Technology, and to **FlowGenX AI**, for jointly making this six-month internship possible.

I am deeply grateful to the academic faculty of IICT, SUST, especially **Dr. Mohammad Reza Selim**, the *Honourable Director* of IICT, and **Mr. Mahfuzur Rahman Emon**, Lecturer, for coordinating the internship placement programme and giving students like me the opportunity to gain hands-on industry experience before graduation.

I owe my deepest and most personal thanks to **Mr. Sumon Saha**, **Founder & CEO of FlowGenX AI**. Sumon Sir did not simply offer me an internship; he offered me a front-row seat — the rare privilege of working directly alongside him and the brilliant senior engineers he has gathered around him. He opened door after door of opportunity that an intern in my position could not have hoped for, gave me real responsibility paired with the flexibility to grow into it, and personally took the time to teach me modern engineering practices I had not encountered in academic life. Despite the relentless demands on his calendar, he never once made me feel that any of my questions were too small, heard me out patiently, and never spoke a single unkind word in any of our interactions. That kind of human care, extended by someone of his stature to an intern, is something I will carry with me through the rest of my career.

I am thankful to **Mr. Balaji Sundara**, *Vice President of GTM and Growth*, for the strategic guidance that gave context to the larger picture of the work I was contributing to.

I extend my deepest appreciation to my immediate supervisor, **Mr. Abrar Bin Rofique**, Software Engineer (Gen AI), whose day-to-day mentorship, code reviews, and patient explanations were central to my growth; his insistence on quality over speed shaped the way I now think about software engineering. I am equally thankful to my additional mentor **Mr. Ashikuzzaman Abir**, Software Engineer, who patiently walked me through how a connector becomes a callable node inside an end-to-end agent workflow whenever my work crossed that boundary.

My sincere thanks go to my **Team Lead Mr. Sanjib Sen**, *Senior Software Engineer*, for his architectural guidance and trust in assigning me production-grade work; and to my closest teammates **Mr. Nayem Islam** and **Mr. Latiful Kabir**, *Software Engineers*, who collaborated with me daily on the connectivity team and made remote teamwork feel genuinely warm.

Finally, I thank every member of the FlowGenX AI engineering, product, and operations teams whose collective effort makes the platform what it is today.

Md. Rakibul Kabir

Executive Summary

This report summarises my internship at **FlowGenX AI**, a San Francisco-based software company building an *Agentic Integration Platform* for designing, orchestrating, and deploying AI agents and workflows through a single no-/low-code interface. The internship was completed as the partial fulfilment of the SWE–420 Internship course of the Bachelor of Science in Software Engineering programme at IICT, Shahjalal University of Science and Technology, Sylhet.

I worked at FlowGenX AI as a *Full Stack Software Engineer — Generative AI (Intern)* from **10 November 2025** to **02 May 2026**, full-time and remotely from Bangladesh. From the first week I was placed on the connectivity team and trusted with production-grade work that ships to real users on the platform.

The bulk of my contribution went to the `runtime_connectivity` project — FlowGenX's unified Python framework that exposes more than two hundred external services through a single standardised interface. Over five and a half months I authored **178 commits** and delivered **more than ninety production-ready connectors**, each implemented end-to-end against the project's three-tier abstraction, validated by dedicated integration test scripts, and accompanied by auto-generated metadata. Beyond feature work I shipped reliability fixes (OAuth 2.0 refresh, retry logic, encoding fixes) and authored UI Endpoint Test Reference documentation for many high-value connectors.

The internship also exposed me to the engineering culture of a globally distributed startup — a disciplined feature-branch Git workflow, peer-reviewed pull requests, automated code-quality enforcement, and asynchronous collaboration across the United States and Bangladesh time zones — and gave me practical exposure to FastAPI, Apache Airflow, the Model Context Protocol (MCP), and the Agent-to-Agent (A2A) patterns at the heart of FlowGenX's agentic stack. The remaining chapters present the company profile, the engineering process, the internship activities, the skills acquired, remote-work reflections, and a concluding reflection.

Contents

Chapter 1

Introduction

1.1 Origin of the Report

The Bachelor of Science in Software Engineering programme offered by the Department of Software Engineering of the Institute of Information and Communication Technology (IICT), Shahjalal University of Science and Technology (SUST), requires every final-year student to complete a six-month industry internship under the course code **SWE–420**. The course is an eighteen-credit lab course that bridges the gap between classroom learning and real-world software engineering practice, exposing students to the technologies, processes, teams, and culture they will encounter as professionals.

This report has been prepared as the formal academic deliverable of that internship. It documents my experience at **FlowGenX AI**, a software development company headquartered in San Francisco, California, that builds an agentic integration platform combining multi-agent orchestration, low-code workflow design, and a large catalogue of native connectors. The document was prepared under the guidance of my academic department and the company's engineering leadership.

1.2 Objectives of the Report

The internship programme, and by extension this report, has both an academic and a personal objective. The specific goals I set for myself during the internship and that this report aims to demonstrate are:

1. **Bridging theory and practice.** To translate the theoretical concepts of software engineering acquired across seven semesters — programming languages, databases, software architecture, software engineering principles — into hands-on, production-grade contributions on a real product.
2. **Learning industry-standard engineering practices.** To work inside a disciplined feature-branch Git workflow, write code that passes peer review, build and run automated test suites, and contribute to a continuous integration pipeline used by paying customers.
3. **Working on a meaningful, production-grade project.** To take complete ownership of features — in my case, end-to-end connector development — from API research and design through implementation, testing, documentation, and live deployment.
4. **Developing depth in modern Generative AI engineering.** To gain practical exposure to large language model providers, the Model Context Protocol (MCP), Agent-to-Agent (A2A) communication, Retrieval-Augmented Generation (RAG) knowledge bases, and the workflow engines that compose these primitives into real applications.
5. **Building professional non-technical skills.** To grow as a remote-first engineer collaborating across time zones with a globally distributed team, communicating clearly through

text and video, planning sprint work, raising blockers early, and giving constructive code review to peers.

6. **Understanding modern startup engineering culture.** To experience first-hand how a small, ambitious team builds a serious product that competes with established platforms in the integration space.
7. **Documenting the experience faithfully.** To produce a report that honestly reflects the work, the learnings, and the challenges of the internship — so that it serves as both an academic deliverable and a record I can return to in my own future.

1.3 Scope of the Report

This report describes the company, the engineering process, the projects I worked on, and the skills I acquired during my internship at FlowGenX AI between November 2025 and May 2026. Specifically, the report covers:

- An overview of the internship programme as it was structured at FlowGenX AI: the team I was placed on, the supervisor and teammates I worked with, the project I contributed to, and the duration and weekly cadence of the work.
- A profile of FlowGenX AI as an organisation — its products, its agentic platform architecture, its target market, the technologies it uses, the office environment (including the realities of remote work), and the company culture I experienced.
- The software engineering methodology and process I followed at the company: the development lifecycle, branching strategy, code review and testing practices, the connector development pipeline, and the tools used across version control, project management, code quality, and deployment.
- A detailed account of my actual contributions, organised in two phases — an initial onboarding and learning period followed by sustained live project work on FlowGenX's `runtime_connectivity` project. This includes specific connectors I designed and shipped, bugs I fixed, refactoring I drove, and documentation I authored.
- The technical and non-technical skills I acquired during the internship and the personal reflections that came out of working inside a remote-first global startup.

The report focuses only on what falls within the scope of an academic deliverable. It is not a technical specification of FlowGenX's product, nor an exhaustive engineering write-up of every commit; it is a faithful reflection of an intern's six-month journey inside the company.

1.4 Methodology

The information presented in this report has been compiled from multiple complementary sources to ensure both accuracy and personal authenticity:

- **Direct work experience.** The bulk of this report is drawn directly from my five and a half months of full-time work inside the company — the projects I shipped, the meetings I attended, the people I worked with, and the artefacts I produced.

- **Version control history.** For Chapter 4 in particular, I systematically reviewed my Git commit history on the `runtime_connectivity` repository to compile an accurate timeline of what was delivered when, including branch names, merged pull requests, and per-period focus areas.
- **Internal repository documentation.** I referred to the project's internal documentation files (including its onboarding-and-conventions guide for AI-assisted code editors) to describe the engineering process, architecture, and conventions accurately.
- **Public-facing company materials.** Company background, vision, product positioning, platform pillars, and use cases were drawn from the company's public website (`https://www.flowgenx.ai/`, including the *About*, *Platform*, *Use Cases*, and *FAQ* pages).
- **Direct communication with company leadership.** For factual gaps that the public website does not address (such as employee count and team composition), I have where appropriate reached out to the leadership team for confirmation rather than speculate.

1.5 Limitations

Although this report has been prepared with care, three limitations should be acknowledged upfront:

1. **Confidentiality and intellectual property constraints.** My internship offer letter establishes a strict Intellectual Property and Confidentiality clause. As a result, proprietary internal architecture, business logic, customer data, pricing details, and unreleased product roadmap items have been deliberately excluded from this report. Wherever such details are relevant, only the publicly disclosed or genuinely non-sensitive aspects are described.
2. **Personal vantage point.** The narrative is written from my own perspective as an intern on a single team — the connectivity team — working on a single project (`runtime_connectivity`). FlowGenX AI is a multi-team organisation building a multi-product platform; my view of the rest of the company is therefore second-hand and necessarily incomplete.
3. **Remote-first nature of the engagement.** Because the internship was fully remote, the chapter on *Life at FlowGenX AI* (Chapter 6) does not describe an in-office experience the way a co-located internship would. It instead reflects what daily life looks like as a remote engineer inside a globally distributed team — an experience that is increasingly common in the modern software industry.

Within these honest limitations, this report aims to be a complete and genuine reflection of my internship at FlowGenX AI.

Chapter 2

Company Profile: FlowGenX AI

This chapter introduces **FlowGenX AI** as an organisation: its products, its agentic platform architecture, the technologies that power it, the use cases it targets, and the working environment it offers its employees and interns. The contents of this chapter are drawn from FlowGenX AI's public website (`https://www.flowgenx.ai/`), the internal repositories I worked on during the internship, and direct conversations with the engineering leadership.

2.1 Company Overview

FlowGenX AI is a software development company headquartered in **San Francisco, California** and founded in **2024**, that builds a next-generation *Agentic Integration Platform*. The platform allows enterprises to design, orchestrate, and deploy AI agents and workflows from a single, no-/low-code interface — bringing together multi-agent orchestration, native protocol-level integration with hundreds of external services, a vector-powered knowledge base, and enterprise-grade security and governance into one coherent product.

The company describes itself with the tagline *"Build Agentic Workflows. Automate Any Process, Integrate Any App"* and positions its product as *"one platform to design, orchestrate, and deploy AI agents and workflows — from idea to production in minutes"*. In the broader software market, the platform sits at the intersection of two historically separate categories: *integration platform as a service* (iPaaS), which has historically been served by tools such as Zapier, Make, n8n, and Workato; and the newer category of agentic AI orchestration frameworks. FlowGenX AI's positioning is to unify these two worlds: every integration becomes an intelligent tool that agents can discover, reason about, and use.

FlowGenX AI is registered as a corporate entity in **California, United States**, and operates as a globally distributed organisation of approximately **seventeen** people, of whom **thirteen** are engineers based in Bangladesh. The company does not currently maintain a separately registered entity in Bangladesh; the Bangladesh-based engineers and interns are engaged as remote team members of the United States organisation. The company is currently **bootstrapped**. Beyond Engineering, FlowGenX AI's organisation includes contributors covering *Product Management*, *Product Marketing*, *Go-to-Market and Business Development*, and *Field-CTO* support based in India, and works with a third-party agency for lead-generation activities. The team collaborates remotely through Google Chat and Google Meet across the Dhaka, San Francisco, and India time zones.

2.2 Vision

The company's vision, taken verbatim from its *About* page, is:

> "FlowGenX AI envisions a world where enterprise integration is transformed through intelligent agentic orchestration. We break down data silos, automate complex workflows, and enable seamless integration between legacy systems and modern cloud platforms — all through our intuitive platform that puts the power of AI-driven integration into the hands of both technical and non-technical users."

This vision shapes everything about how the platform is built. Rather than treating integration as a developer-only activity (the way classical iPaaS tools do) or treating agentic AI as a research toy (the way many open-source agent frameworks do), FlowGenX AI deliberately blurs the line between the two. Workflows are first-class citizens; so are agents; and connectors are the substrate through which both interact with the rest of the world.

2.3 What FlowGenX AI Does

At a high level, FlowGenX AI helps enterprises do three things:

1. **Connect any system to any other system.** Through a catalogue of more than two hundred natively built connectors plus the ability to import any external API and auto-convert it into a Model Context Protocol (MCP) tool, the platform can talk to databases, cloud storage, SaaS applications, AI/LLM providers, helpdesks, CRMs, ERPs, and almost any other system an enterprise runs.
2. **Compose those systems into intelligent workflows.** A visual, drag-and-drop workflow builder lets technical and non-technical users alike compose multi-step processes that move data, transform it, apply privacy policies, branch on conditions, iterate over collections, and call external services.
3. **Add agentic intelligence on top of those workflows.** Multi-agent orchestration patterns (ReAct, Supervisor, Swarm, and Deep agent patterns) coordinate multiple AI agents that reason about the next step, distribute tasks among themselves, and collaborate in real time, all while remaining inside the same governed control plane as the deterministic workflow steps.

This is captured in the company's promise of being a *"Complete Agentic Stack"*: agent orchestration, MCP-native integrations, A2A communication, vector-powered knowledge base, and a no-code workflow builder, all governed by a single control plane.

2.4 The FlowGenX Platform

The platform is organised around six functional pillars. Together they form what FlowGenX calls the *Complete Agentic Stack*.

2.4.1 Multi-Agent Orchestration (Agent Fabric)

Agent Fabric is the first-class agentic runtime of the platform. It supports the most widely used multi-agent design patterns out of the box — including **ReAct**, **Supervisor**, **Swarm**,

and **Deep** agent patterns — and provides the supporting infrastructure underneath them: an agent registry, a task broker, agent-to-agent (A2A) communication, policy evaluation, and distributed observability traces. Multiple agents can reason about the next step, distribute tasks between themselves, and collaborate in real time. Agent execution is observable, governable, and supports human-in-the-loop approval gates at any step.

2.4.2 Agentic Integration Engine (Integration Studio)

The integration substrate of the platform. It exposes more than **200 managed connectors** for databases, cloud storage, software-as-a-service applications, and AI providers. In addition, arbitrary external APIs can be imported and *automatically converted into MCP tools* that any agent can discover and call. This is the layer my own work — the `runtime_connectivity` project — sits inside.

2.4.3 Workflow Development

A visual, drag-and-drop workflow builder where users compose multi-step processes graphically. Workflows can be triggered by events, run as batch jobs, or invoked on demand. They support inline data transformation, privacy policies, conditional branching, iteration, and full traceability across every step. Figure 2.1 shows the workflow editor.

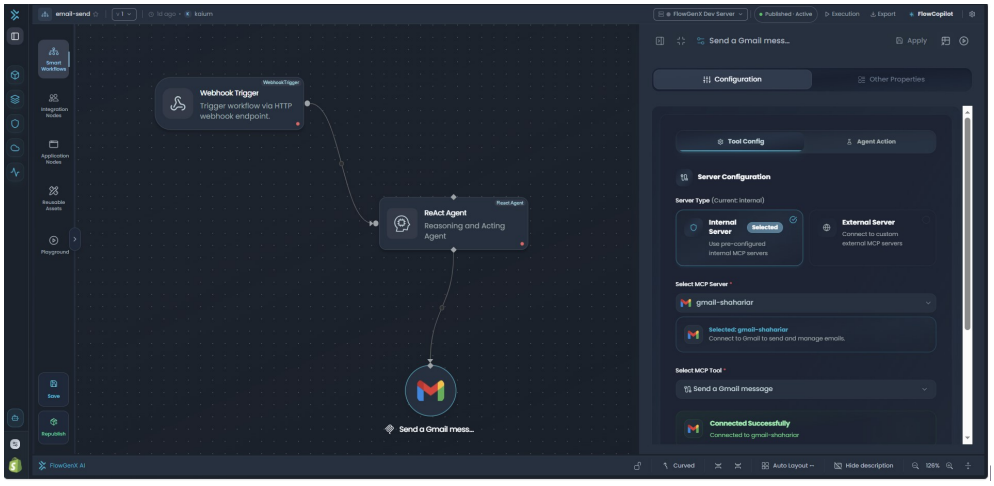

Figure 2.1: The FlowGenX AI visual workflow editor.

2.4.4 Intelligent Knowledge Base (VectorVerse)

VectorVerse is the platform's enterprise knowledge-base product: a vector-powered Retrieval-Augmented Generation (RAG) layer that brings together document ingestion, semantic search, collections, agent memory (episodic, semantic, and procedural), explainability tracing, observability metrics, and governance controls. It provides agents with grounded enterprise context so that answers remain anchored in real source material rather than hallucinated, and it exposes a sandbox where engineers can test queries against a knowledge collection before wiring it into a workflow.

2.4.5 Work AI

A suite of higher-level AI capabilities that sit on top of the agent fabric and the workflow engine, intended to give end users a copilot-style experience for everyday automation tasks.

2.4.6 Security & Governance (Agent Gateway)

Agent Gateway is the enterprise-grade security and governance layer that sits in front of every API, MCP tool, and agent on the platform. It enforces multi-tenant context (tenant, environment, and principal headers), authentication and authorisation, policy evaluation, rate limiting, task-approval workflows, audit logging, and trace observability. Agents act autonomously by default but always under the constraints set by the gateway, and human-in-the-loop approval gates can be inserted wherever judgement is required.

2.5 Other Notable Platform Components

Beyond the six headline pillars, the platform exposes a number of supporting components that I encountered while working on the connectivity layer and that are worth describing here because they shape how end users actually consume the platform.

2.5.1 Playground

The *Playground* is the platform's interactive testing surface. It provides three modes — *MCP*, *Agent*, and *Workflow* — so that engineers and end users can exercise an MCP tool, talk to an agent, or step through a workflow without having to deploy it. The MCP Playground in particular (Figure 3.1) was a tool I used routinely during connector development to validate that each newly shipped connector appeared correctly as an MCP tool and that its parameters and responses matched the OpenAPI specification.

2.5.2 Webhook Trigger

A *Webhook Trigger* is one of the workflow's entry points: an HTTP endpoint that initiates workflow execution when an external system posts data to it. The trigger supports configurable authentication (none, API key, OAuth), request-payload validation, custom headers, CORS, rate limiting, and synchronous or asynchronous response modes. It is what allows FlowGenX workflows to be invoked from any third-party system that can fire an HTTP webhook.

2.5.3 API Workspace

The *API Workspace* is the unified management interface for APIs and tools on the platform. It is where engineers and administrators import OpenAPI specifications, define new API-based tools, manage their authentication credentials, test endpoints, and expose them for consumption inside workflows. The OpenAPI metadata generated by the

`@openapi_endpoint`-decorated source code in `runtime_connectivity` ultimately surfaces in the API Workspace.

2.5.4 Data Compiler

The *Data Compiler* (implemented as the schema-drift component in `runtime_components`) is the platform's defence against upstream schema change. As data flows through a workflow it continuously validates incoming payloads against the expected target schema, detects drift in column types, missing fields, and format deviations, and applies AI-driven corrections through a SchemaDriftAgent before the data reaches the downstream component. This makes long-running pipelines resilient to the kind of silent upstream changes that would otherwise cause a workflow to fail mid-execution.

2.6 Use Cases and Target Industries

FlowGenX AI's public use-case catalogue covers eleven distinct real-world scenarios across seven enterprise functions. Table 2.1 summarises them.

Function	Use Case	What it does
Operations	Document Intelligence	Extracts and reconciles data from PDFs, Excel, Word, and images, and pushes it into ERPs and CRMs.
Operations	Supply Chain Operations	Monitors inventory, flags disruptions, and coordinates with suppliers autonomously.
IT	IT Support Desk	Resolves employee requests by triaging, routing, and closing tickets without human intervention.
IT	IT Ticket Triage	Classifies, prioritises, and assigns tickets in real time based on context and team capacity.
Sales	CRM Enrichment	Continuously researches and enriches contacts and accounts from the public web, SEC filings, and business databases.
Sales	RFP Drafting	Researches requirements and drafts complete RFP responses in minutes rather than days.
Compliance	Compliance Monitoring	Monitors regulatory changes and surfaces internal-policy risks before they become violations.
Finance	Financial Reporting	Aggregates, reconciles, and formats board-ready financial reports on a recurring schedule.
HR	HR Onboarding Automation	Provisions accounts, sends welcome materials, and guides new hires through their first day.
Marketing	Email Campaign Operations	Segments audiences, personalises content, and tracks performance across multi-step campaigns.
Capability	Web App Automation	Drives real browsers through enterprise web apps using structured DOM perception.

Table 2.1: FlowGenX AI's published use cases across seven enterprise functions.

The breadth of these use cases reflects the platform's positioning: the same underlying primitives (agents, workflows, connectors, knowledge base) can be assembled to solve very different business problems across operations, IT, sales, compliance, finance, HR, and marketing.

2.7 Tools and Technologies Utilised by FlowGenX AI

FlowGenX AI has deliberately chosen a modern, polyglot, cloud-native technology stack. The choices are visible in the codebases I worked on directly.

2.7.1 Programming Languages

- **Backend:** Python 3.11+, used across the connectivity framework, the workflow APIs, and the reusable component library.
- **Frontend:** TypeScript and JavaScript, on top of React and Next.js.
- **Configuration and data exchange:** JSON, YAML, and OpenAPI 3.0 specifications.
- **Database queries:** SQL, used to query the platform's own PostgreSQL metadata store and to implement the database-family connectors.

2.7.2 Backend Frameworks and Libraries

- **API framework:** *FastAPI* for the runtime APIs.
- **Workflow engine:** *Apache Airflow* for pipeline scheduling, orchestration, and DAG-style execution of workflows.
- **Data validation and schema modelling:** *Pydantic* (used pervasively for connection parameters, component schemas, and request/response models).
- **Data manipulation:** *pandas* for in-memory DataFrame transformations.
- **Templating:** *Jinja2* for dynamic, runtime- parameterised configuration of components.
- **HTTP and async I/O:** *requests*, *httpx*, and *asyncio*.

2.7.3 Frontend Frameworks and Libraries

- **Framework:** *Next.js* on top of *React* powers the FlowGenX web application where users design workflows, configure connectors, and chat with agents.
- **Runtime:** Node.js.
- **Editor experience:** The Monaco editor is integrated for in-browser code and expression editing.

2.7.4 Platform Data Stores

The platform itself runs on a deliberately small set of data stores. These are the databases that hold FlowGenX's own state — not the many third-party data sources that users can connect through the integration catalogue (those are described separately in Section 2.8).

- **Primary metadata store:** *PostgreSQL*. Holds the connector registry, authentication configurations, workflow definitions, and other operational state.
- **Vector store for the Knowledge Base feature:** the Knowledge Base pillar of the platform stores document embeddings in a vector database (with support for multiple backends such as *Pinecone*, *Weaviate*, and *Milvus*) so that Retrieval-Augmented Generation (RAG) queries from agents can return grounded, source-cited answers.

2.7.5 Cloud Infrastructure and DevOps

- **Cloud providers (for hosting the platform itself):** Amazon Web Services and Microsoft Azure are the primary deployment targets.

- **Containerisation and orchestration:** *Docker* for containerisation, *Kubernetes* for orchestration, and *Helm* for Kubernetes packaging.
- **Infrastructure as Code:** *Terraform*.
- **API specifications:** *OpenAPI 3.0* (Swagger), auto-generated from the connector source code.

2.7.6 Agentic Platform Primitives

The agentic stack of the platform is built on a small set of design patterns and protocols rather than on any single LLM vendor. Specific large language models are accessed *through the integration catalogue* (Section 2.8) — they are interchangeable tools, not part of FlowGenX's own technology stack.

- **Multi-agent orchestration patterns:** *ReAct*, *Supervisor*, *Swarm*, and *Deep* agent patterns, all supported as first-class primitives.
- **Model Context Protocol (MCP):** an open protocol used to expose every connector as a standardised tool that any agent can discover and call. Arbitrary external APIs can also be imported and *auto-converted* into MCP tools.
- **Agent-to-Agent (A2A) protocol:** used for discovery and synchronous or asynchronous communication between agents.
- **Vector-RAG pipeline:** the Knowledge Base feature combines document ingestion, semantic chunking, embedding, vector search, and source citation to ground agent responses in real source material.

2.7.7 Engineering Tooling

- **Version control:** Git, hosted on GitHub.
- **Python package management:** *uv*.
- **Code formatting:** Black (with a 79-character line length).
- **Import sorting:** isort, configured with the *black* profile.
- **Linting:** Ruff, with selected rule sets.
- **Pre-commit hooks:** enforce all of the above automatically before each commit.
- **Project management:** ClickUp.
- **Communication:** Google Chat and Google Meet.

2.8 Integration Catalogue

A defining feature of the FlowGenX platform is its catalogue of more than **two hundred natively built connectors** that customers can use to integrate third-party services into their workflows. It is important to draw a clear conceptual line: *the catalogue is not part of FlowGenX's own technology stack.* The technologies listed in Section 2.7 are what FlowGenX engineers use to *build* the platform. The integrations listed here are what *customers* can choose to plug *into* the platform when they design their own workflows.

Each connector is implemented as a small, self-contained plug-in inside the company's connectivity layer (the `runtime_connectivity` package described in Section 2.9) and exposed through the platform UI as a configurable building block. End users decide which connectors to authenticate, which methods to call, and how to compose them inside their workflows; FlowGenX engineers maintain the catalogue itself, but no individual connector is required for the platform to function.

The catalogue spans **databases** (PostgreSQL, MySQL, SQL Server, Snowflake, Azure SQL, AWS RDS, MongoDB, Elasticsearch, Typesense, Pinecone, Weaviate, Milvus); **cloud storage** (S3, Azure Blob, Azure Data Lake, OneDrive, SharePoint); **AI/LLM providers** (OpenAI, Anthropic, Gemini, Vertex AI, xAI, Groq, DeepSeek, Perplexity, AWS Textract, AWS Rekognition); **search and research data** (Tavily, Exa, SerpAPI, OpenFIGI, FactSet, S&P Global, Crunchbase, AlphaSense, Octagon); **CRM and sales** (Apollo.io, Attio, Outreach, SalesLoft, Gong, Chorus, 6sense, LinkedIn Sales Navigator, Zoho CRM, Zendesk Sell, Zoho Desk); **finance and accounting** (Stripe, QuickBooks, Xero, FreshBooks, Zoho Books, Zoho Inventory, Dynamics 365 BC, dbt-cloud); **marketing** (Mailchimp, Customer.io, Instantly, NeverBounce, Zoho Campaigns); **productivity and collaboration** (Notion, ClickUp, Linear, Discord, Zoom, Google Meet/Chat/Forms/Tasks, Typeform, Eventbrite, Supabase); **helpdesk** (FreshDesk, Freshservice, HelpScout, Jira Service Management); **HR and payroll** (BambooHR, Lever, Ashby, Rippling, Gusto); **DevOps** (GitHub, GitLab, GitHub Actions, Harness, SonarQube, LaunchDarkly, Datadog); **identity** (Auth0, Okta); **healthcare** (EPIC FHIR); **social and commerce** (Twitter/X, LINE, WordPress, Shopify, Yelp, Uber); and **other utilities** (Apify, OpenWeatherMap, Massive).

Every connector implements one or more of the standard authentication patterns used by modern APIs — OAuth 2.0 with refresh-token flows, API key or bearer-token authentication, HTTP basic authentication, and AWS IAM credential flows. The detailed account of which connectors I personally designed, implemented, and shipped during the internship is given in Chapter 4.

2.9 Internal Architecture (At a High Level)

The platform is organised across four primary internal repositories, each with a clear single responsibility:

- `flowgenx-nextjs-V2` — the customer-facing web application built on Next.js and React. This is what users interact with when they design workflows, configure connectors, or chat with their agents.
- `runtime_apis` — the FastAPI service that exposes workflow execution and component management endpoints, backed by Apache Airflow as the underlying workflow engine and PostgreSQL for state. The repository describes itself as *"a flexible and scalable data pipeline workflow platform that combines Apache Airflow's workflow management capabilities with FastAPI's modern API development features"*.
- `runtime_components` — a reusable Python library of pipeline components used by the workflow engine. It currently ships ten or more pre-built components, including `IngestionComponent`, `FilterComponent`, `DataMappingComponent`, `DataPrivacyComponent` (with built-in masking and anonymisation for credit-card numbers, SSNs, addresses, email,

and other PII), `WriteComponent`, `JoinerComponent`, `HTTPNodeComponent`, `ForeachComponent`, `SwitchComponent`, and `SensorComponent`. All components are configured through Pydantic-validated schemas and can be parameterised at runtime through Jinja2 expressions.

- `runtime_connectivity` — the unified connectivity framework I contributed to, which exposes external services through a single standardised interface. This repository is the subject of Chapter 4.

2.10 Office Environment and Working Style

FlowGenX AI operates as a remote-first organisation with engineers based in the United States and Bangladesh. My own internship was a fully remote engagement from Dhaka, working on my personal laptop.

- **Communication tools.** The team uses Google Chat for asynchronous messaging and Google Meet for synchronous discussions and daily standups. Microsoft Teams was used previously and was migrated away from during my tenure.
- **Project management.** Work is tracked on *ClickUp* boards, with each connector and feature represented as a card with status, assignee, and links to the relevant pull request.
- **Working hours.** I generally worked from approximately 9:00 a.m. to 5:00 p.m. Bangladesh Standard Time (UTC+6), with flexibility to shift hours when needed to overlap with the United States team for synchronous meetings.
- **Cadence.** The team operates on a **weekly sprint cadence** (a hybrid of Scrum and Kanban practices) with daily asynchronous updates and synchronous reviews when needed. Production rollouts are typically performed on Saturdays.

2.11 Company Culture

The cultural traits I directly experienced over five and a half months at FlowGenX AI are:

- **Ownership and autonomy.** From my first connector onwards, I was trusted to design and ship features end-to-end — research the third-party API, design the connection-parameter schema, implement the manager class and methods, write integration tests, generate the metadata, open the pull request, address review feedback, and merge.
- **Mentorship over micromanagement.** My supervisor Mr. Abrar Bin Rofique reviewed every pull request thoroughly and explained the *why* behind every suggestion, but never micromanaged the *how*. The same was true of my team lead.
- **Innovation and curiosity.** Engineers are encouraged to learn and to bring back new patterns. During my internship I was given room to explore the Model Context Protocol, the Agent-to-Agent protocol, and the workflow engine's internals — all beyond the strict scope of my connector work.
- **Continuous learning.** The company maintains an extensive blog (`https://blog.flowgenx.ai/`) and developer documentation (`https://docs.flowgenx.ai/`) that engineers are encouraged to read and contribute to.

Chapter 3

Software Methodology and Engineering Process

This chapter describes the software engineering process I followed during my internship at FlowGenX AI: the development methodology, the end-to-end development lifecycle of a feature, the version-control and code-review workflow, the testing strategy, the metadata-generation pipeline that takes a connector from source code to a live UI, and the supporting toolchain.

The descriptions in this chapter are drawn from the project's internal documentation and from my own day-to-day experience working inside the process for five and a half months.

3.1 Development Methodology

FlowGenX AI follows a hybrid engineering process that combines **Scrum** and **Kanban** practices, organised around a **weekly sprint cadence**. Work is tracked on a *ClickUp* board where each connector or feature is represented as a card with status, assignee, and links to the corresponding feature branch and pull request. Tasks move through a typical kanban-style flow — *To Do*, *In Progress*, *In Review*, *Implemented*, *API Tested*, *MCP Playground Tested*, and *Workflow Tested* — giving the team end-to-end visibility into where every feature is in its lifecycle.

In practice, the process I experienced has these defining characteristics:

- **Incremental delivery.** Features are shipped as small, self-contained units (typically one connector at a time), each on its own branch and merged independently.
- **Asynchronous-first communication.** Most coordination happens through Google Chat threads and ClickUp comments. Live meetings are reserved for design discussions, code-walkthroughs, and blockers.
- **Reviewed everything.** No code reaches `dev` without a peer-reviewed pull request.
- **Automated enforcement of conventions.** Black, isort, Ruff, and pre-commit hooks reject style violations before code is even pushed.
- **Documentation as a deliverable.** A connector is not considered complete until it has both an integration test script and a UI Endpoint Test Reference document.

3.2 End-to-End Development Lifecycle

The lifecycle of a feature at FlowGenX AI — specifically a new connector, which was my primary unit of work — moves through eight distinct stages.

3.2.1 Requirement Gathering

A new connector is added to the platform either because a customer or prospect needs it, or because the team is expanding the catalogue strategically. Requirements are typically expressed as a ClickUp card with the third-party platform name, its category (database, CRM, finance, AI provider, and so on), the priority methods to expose, and links to the third-party API documentation. The ClickUp board for the connector backlog also tracks columns such as *Available on UI?*, *Auth Type*, *Implementation Status*, *API Tested*, *MCP Playground Tested*, and *Workflow Tested*.

3.2.2 Design

Before writing code, I would spend time reading the third-party API documentation and identifying:

- The authentication pattern (API key, OAuth 2.0, basic auth, AWS IAM, and so on).
- The list of methods to expose (often dictated by what real workflows need).
- The pagination and rate-limiting model.
- The shape of request and response payloads.
- Any quirks (regional endpoints, multi-tenancy, async operations, file uploads).

This research output translates directly into the design of three artefacts: the `ConnParams` Pydantic schema, the `Manager` class, and the per-method modules.

3.2.3 Implementation (the Development Cycle)

A new connector lives under `src/runtime_connectivity/connectors/<name>/` and follows a standardised three-tier architecture. **Base managers** (defined once in `connectors/base.py`) provide the abstract `BaseManager` foundation plus three concrete bases: `DBManager` for relational databases (with `_list_tables`, `_read_table_as_dataframe`, `_execute_query_to_database`, `_write_table_to_dataframe`, `_get_table_schema`); `FSManager` for file systems (`_list_files`, `_read_file`, `_write_file`, `_delete_file`); and `ObjManager` as the catch-all base for object/API-based connectors. The **connector implementation** layer adds an `__init__.py` that exposes the connector's `ConnParams` Pydantic model, its `Manager` class, and its per-method modules; per-method files (for example `create_customer.py`) decorated with `@openapi_endpoint`; a `test_connection.py`; and (for newer connectors) a `specs/` folder with the auto-generated metadata. Finally, the **connector registry** in `connectors/__init__.py` wraps registration in a `try/except` so a single broken connector cannot break the package import for everyone else.

Every public method must be decorated with `@openapi_endpoint`, which attaches the OpenAPI metadata (HTTP method, path, summary, parameters, request and response schema, tags) that the platform later consumes to render the UI form. To support both synchronous and asynchronous callers, all methods are wrapped through `dual_method_support_from_async` (used inside FastAPI handlers and other async contexts) or `dual_method_support_from_sync` (used outside event loops; raises if called from inside one).

3.2.4 Code Review and Pull Requests

Once a connector is implementation-complete, I would push the feature branch and open a pull request against `dev`. Pull requests follow a disciplined commit-message convention, `feature/Rakib/(<connector>): <summary>`, so reviewers can trace work per integration. Each PR is reviewed by at least one senior engineer (typically my supervisor Abrar or the team lead Sanjib). Reviewers comment line-by-line, request changes when the `ConnParams` schema needs more validation, when an OpenAPI example is missing, or when the test coverage is insufficient. A typical PR goes through one or two rounds of review before being merged. The team operates a **two-branch pull-request flow**: feature branches are reviewed and merged into the development branch, and a second pull-request flow promotes changes from the development branch into the production branch.

When a teammate's changes had landed in `dev` between the time I branched and the time I was ready to merge, I would rebase or merge `origin/dev` into my branch and resolve conflicts manually — non-trivial conflict resolutions occurred on connectors including AWS Textract, HelpScout, X (Twitter), Harness, Google Forms, FreshBooks, Freshworks, and Zoho CRM.

3.2.5 Testing Strategy

Testing at FlowGenX AI happens at three layers.

Unit and contract tests. Each connector ships with `tests/connectors/test_<name>.py`. These tests use `pytest` fixtures and `unittest.mock` to validate parameter validation, connection establishment, DataFrame read/write operations, and error paths — without making real network calls.

Integration tests against the live API. For every connector I delivered, I additionally wrote a dedicated integration test script under `scripts/` named `test_<connector>_connector.py`. These scripts hit the real third-party API with real credentials and produce a PASS / FAIL / SKIP report per endpoint. This is the single most important quality gate: it is what catches authentication subtleties, pagination edge cases, schema drift in the upstream API, and serialisation issues that mocked tests cannot catch.

End-to-end UI and workflow validation. Once merged, every connector is validated through three additional checks that are tracked on the ClickUp board: an *Implementation* review, an *API Tested* pass, an *MCP Playground Tested* pass (Figure 3.1), and a *Workflow Tested* pass to confirm it actually composes inside a real end-to-end workflow (Figure 2.1).

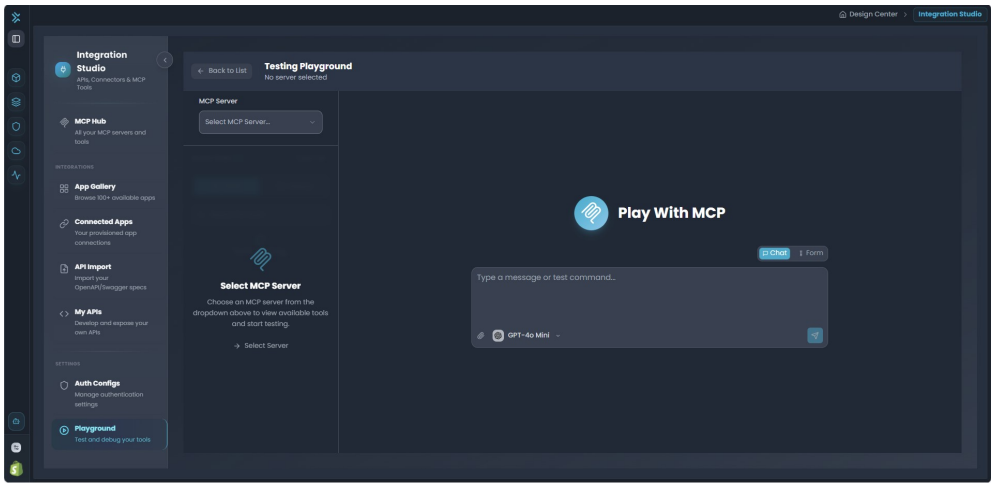

Figure 3.1: MCP Playground for end-to-end testing of connector tools.

3.2.6 Deployment to Development and Production

The platform currently uses two environments, `Dev` and `Prod`, with continuous-deployment pipelines built on **GitHub** and **ArgoCD**. After a pull request is merged into the `dev` branch, the change is deployed to the development environment, where the team performs API testing and end-to-end workflow validation. Promotion to production goes through a second pull-request flow into the production branch and is typically rolled out on **Saturdays** once the release is deemed stable. Validation activities ahead of each rollout are continually being strengthened as the engineering team grows. The platform itself is containerised with Docker and orchestrated with Kubernetes, with infrastructure provisioned through Helm charts and Terraform.

3.2.7 Hotfixes

Urgent production bugs are addressed outside the normal sprint cadence through small, targeted patches that follow the same review-and-test discipline as regular features but with a faster turnaround. The specific hotfixes I contributed during the internship are listed in Chapter 4.

3.3 Development Tools

The lifecycle described above is supported by a small, consistent toolchain. Source code lives in **Git** on **GitHub**, with isolated feature branches scoped per connector and `dev` as the integration branch from which environment promotions flow. Tasks are tracked in **ClickUp**, where each connector card carries its assignee, status, branch and pull-request links, and the four-stage testing checklist (Implementation / API / MCP Playground / Workflow). Code review happens inline on **GitHub Pull Requests**, where at least one approval is required before merging. Python environments are managed with **uv** (`uv pip install -e ".[dev]"`); runtime services are containerised with **Docker** and orchestrated with **Kubernetes** (packaged via **Helm** and provisioned via **Terraform**); continuous deployment

is driven by **GitHub** and **ArgoCD** into the platform's two-tier `Dev`/`Prod` environment topology, with production rollouts performed weekly on Saturdays. API specifications are **OpenAPI 3.0**, auto-generated from the `@openapi_endpoint`-decorated source code through `generate_openapi_spec.py`. The team communicates through **Google Chat** (asynchronous) and **Google Meet** (synchronous); Microsoft Teams was used previously and was migrated away from during my tenure.

3.4 Error Handling and Logging Conventions

The codebase imposes a small but strict convention for error handling and logging:

- Errors raised inside a connector use `ConnectionExecutionError` (from `runtime_connectivity.erro` with a `message`, `component` (set to `self.conn_type`), and `method` parameter.
- Logging goes through the structured `app_logger` from `runtime_connectivity.logger`, with consistent `component` and `method` context, so log lines can be filtered cleanly per connector and per endpoint at runtime.

Chapter 4

Internship Activities and Project Work

This chapter is the heart of the report. It describes what I actually did at FlowGenX AI during my five-and-a-half-month internship: who I worked with, how I onboarded, what I learned in the early weeks, and the specific live-product contributions I made between November 2025 and May 2026. The chapter is organised into two phases, mirroring the natural arc of my internship: an initial learning period followed by sustained live-project work on the company's connectivity layer.

4.1 Overview of My Role

I joined FlowGenX AI on **10 November 2025** in the position of *Full Stack Software Engineer – Generative AI (Intern)*, and worked full-time, remotely from Bangladesh, until **02 May 2026**. This was a tenure of approximately five and a half months — exceeding the four-month minimum stated in my offer letter — during which I was integrated into the company's *connectivity team.*

4.1.1 The Connectivity Team

The team I joined is responsible for the FlowGenX platform's integration substrate: the framework, the connector catalogue, and all the supporting tooling that exposes external services to the runtime, the workflow engine, and ultimately the end user.

The people I worked with most closely on a daily basis were:

- **Mr. Sanjib Sen** — Senior Software Engineer and Team Lead. Sanjib bhai owns the architectural direction of the connectivity layer and is the senior reviewer for non-trivial changes.
- **Mr. Abrar Bin Rofique** — Software Engineer (Generative AI), my direct supervisor. Abrar bhai owned my onboarding, the day-to-day technical guidance, and the bulk of the code review on my pull requests.
- **Mr. Ashikuzzaman Abir** — Software Engineer, additional mentor. Ashik bhai supports the workflow-agent side of the platform, and he was the person I went to whenever my connector work touched the workflow runtime — helping me understand how a connector method becomes a callable node inside a real end-to-end agent workflow.
- **Mr. Nayem Islam** — Software Engineer / AI Engineer working closely with the connectivity team.
- **Mr. Latiful Kabir** — Software Engineer (Generative AI), also working closely with the connectivity team.

I was the only intern on the team for the duration of my engagement and was treated as a contributing member rather than a learner-only, in keeping with the company-wide culture of valuing impact over titles.

4.1.2 The Primary Project

My primary project was the `runtime_connectivity` repository — FlowGenX AI's unified Python framework for connecting the platform to external systems. By the end of my internship I had authored **178 commits** across the `dev` branch and dozens of feature branches, and delivered **more than ninety production-ready connectors** to the platform's integration catalogue.

4.2 Phase One: Initial Learning Period (Weeks 1–2)

The first two weeks of my internship were a deliberate ramp-up period. The work in this phase was not merely tutorial-style learning; the very first connectors I built (Weaviate, Discord, Notion, and a fix to Google Chat) shipped to production. The intent of the phase was to learn the framework by building inside it.

4.2.1 Setting Up the Development Environment

The very first task was to bring up a working local development environment for the connectivity package and the surrounding runtime services. This involved:

- Installing Python 3.11 and the `uv` package manager.
- Cloning the four repositories (`flowgenx-nextjs-V2`, `runtime_apis`, `runtime_components`, `runtime_connectivity`) into a shared parent directory so that local editable installs could cross-reference each other.
- Running `uv pip install -e ".[dev]"` inside `runtime_connectivity` to install the package in editable mode along with all developer dependencies.
- Installing the `pre-commit` hooks so that every commit I made would be automatically formatted and linted.
- Configuring a local PostgreSQL instance and Docker Desktop on Windows for connector testing and end-to-end integration runs.
- Standing up the FastAPI service through the project's `dev.sh up` script and verifying that the Swagger UI (`http://localhost:8081/docs`) and the Airflow UI (`http://localhost:8080`) were both reachable.

4.2.2 Understanding the Connector Framework

Before writing my first connector, my supervisor walked me through the project's three-tier abstraction (`BaseManager` → `DBManager`/`FSManager`/`ObjManager`), the `@openapi_endpoint` decorator, the dual sync/async pattern, the `ConnParams` convention, and the metadata-generation pipeline that takes a connector from source code to UI. I read the project's internal documentation end-to-end, then reverse-engineered an existing connector (PostgreSQL) to see how all the pieces fit together in practice.

The most important conceptual insight from this period was that a connector at FlowGenX is not just a Python class; it is a *four-artefact bundle*:

1. Source code (`ConnParams` + `Manager` + per-method modules, decorated with OpenAPI metadata).
2. Integration tests (`test_<name>_connector.py`).
3. Auto-generated metadata (`openapi_spec.json`, `auth_config.json`, `connector_metadata.json`).
4. A UI Endpoint Test Reference document for end users.

Internalising this four-artefact contract was what made my work self-consistent from the very first connector onwards.

4.2.3 First Connectors as Learning Artefacts

With my environment set up and the framework understood, the second half of the onboarding period was hands-on. Between **20 November and 30 November 2025** I shipped my first production connectors:

- **Weaviate** — a vector-database connector. This was my first end-to-end implementation. Building it taught me how to model schema-less data, work with vector embeddings, and write the `near_vector` similarity-search method.
- **Discord** — a messaging-platform connector that taught me how OAuth-style bearer tokens work in practice.
- **Notion** — a productivity-tool connector. This is where I first wired up environment-variable support and applied the `@openapi_endpoint` decorator end-to-end.
- **Perplexity AI** — my first AI-provider connector.
- **QuickBooks (initial scaffolding)**, **Uber**, **Shopify**, **S&P Global**, **ClickUp**, **MongoDB**, and an initial **Stripe** connector — each one teaching me a slightly different aspect of authentication, pagination, or error handling.
- A **Google Chat fix** — my first real bug-fix-and-merge experience inside someone else's connector.

By the end of week two I had merged my first half-dozen pull requests and had a complete mental model of the codebase. Phase one was complete; phase two could begin.

4.3 Phase Two: Live Project Involvement

The remaining five months of the internship were spent contributing to `runtime_connectivity` as a normal team member. The work fell into four categories: **feature development** (new connectors), **bug fixes and reliability work**, **code improvement and refactoring**, and **documentation**. I describe each in turn, then narrate a few of the most substantial connectors in greater depth.

4.3.1 The Connector Lifecycle in Practice

Every connector I delivered followed the same eleven-step lifecycle. **(1) API research** — read the third-party documentation and identify the auth model, priority methods,

pagination, rate limits, and quirks. **(2) Branch** the feature off `dev`. **(3) Connection-parameter modelling** — define a `ConnParams` Pydantic model with the right field types, descriptions, and `json_schema_extra` UI metadata. **(4) Manager class** — subclass the appropriate base manager (`DBManager`, `FSManager`, or `ObjManager`) and implement `_connect`, `_close`, and `test_connection`. **(5) Per-method modules** — implement each business method in its own file, decorated with `@openapi_endpoint`, using the dual sync/async helpers. **(6) Integration test script** — `test_<connector>_connector.py` hits the live API with real credentials and produces a PASS /FAIL /SKIP report per endpoint. **(7) Sample backfill** — capture real request and response payloads from the integration run and embed them as the decorator's `example` and `response_sample` values. **(8) Metadata generation** — run `generate_openapi_spec.py`, `generate_auth_config.py`, and `populate_connector_db.py` so the connector becomes visible in the UI. **(9) Pull request and review** — descriptive `feature/Rakib/(<connector>): <summary>` commit, review cycle, merge into `dev`. **(10) Endpoint testing** after merge through the API tester, the MCP Playground (Figure 3.1), and an end-to-end workflow run (Figure 2.1); update the ClickUp card. **(11) Documentation** — for high-value connectors, author a *UI Endpoint Test Reference* cookbook. Internalising and operating this lifecycle cleanly was the single most important habit I built during the internship.

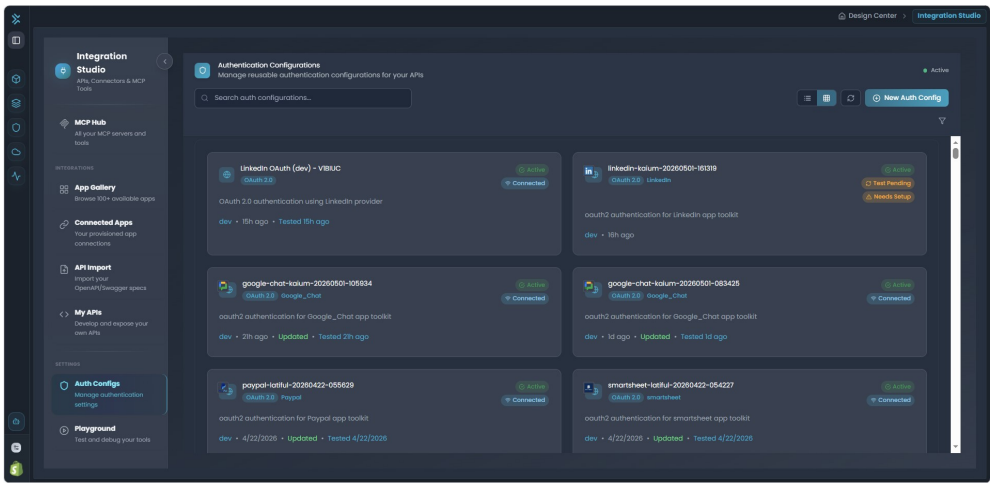

Figure 4.1: Authentication configuration page driven by `auth_config.json`.

4.3.2 Work Timeline

Table 4.1 summarises what I delivered across the twenty-four weeks of the internship. The table is compiled directly from my Git history on the `runtime_connectivity` repository.

Period	Connectors and work delivered
Nov 20 – Nov 30, 2025	Weaviate; Discord; Notion (full setup with env-var support and decorators); Perplexity AI; QuickBooks (initial); Uber; Shopify (test pass); Google Chat fix; S&P Global; ClickUp; MongoDB; Stripe (initial).
Dec 1 – Dec 14, 2025	Elasticsearch (with Windows encoding fix); Datadog; Milvus; LaunchDarkly; Zoho CRM; FreshDesk; Yelp; Stripe; FreshBooks; Xero; GitLab; GitHub; Zoho Desk; Apollo.io; Zoom; WordPress; Customer.io; BambooHR; Lever; OneDrive; SharePoint; FreshBooks endpoint refactor.
Dec 15 – Dec 31, 2025	AWS S3 (bucket policies, versioning, multipart uploads); MySQL upgrades (truncate / update / upsert); PostgreSQL enhancements; Pinecone; Tavily; Exa; SerpAPI; AWS RDS PostgreSQL; AWS RDS MySQL; dbt-cloud; OpenWeatherMap; Freshsales; Zendesk Sell; Auth0; Massive; MSSQL with full CRUD; Azure Blob Storage; Azure SQL Server (firewall + retry); Groq; OpenAI.
Jan 1 – Jan 15, 2026	Anthropic (with Files / Skills API restructure); DeepSeek; xAI; Azure Data Lake; Typesense; X (Twitter); LINE Messaging; Google Tasks; GitHub Actions; Google Forms; Gusto; Zoho Inventory; Zoho Books.
Jan 16 – Jan 31, 2026	Zoho Analytics (User Management + data manipulation); Zoho Campaigns; Mailchimp; EPIC FHIR (initial groundwork and permissioning); Microsoft Dynamics 365 Business Central; Linear; AlphaSense; Gong; Outreach; 6sense; SalesLoft; Rippling; FactSet; Google Chat.
Feb 1 – Feb 28, 2026	Jira Service Management; Typeform; Ashby; Freshservice; Okta; Crunchbase; HelpScout; AWS Textract (23 methods, sync + async + adapters); AWS Rekognition (75 methods covering image, video, face, custom labels, stream processors); HelpScout UI test-reference doc; Rekognition endpoint test reference.
Mar 1 – Mar 15, 2026	Octagon (15 financial-research agents); Harness (46 methods across V1 + legacy APIs); SonarQube (34 methods across server + cloud); QuickBooks access-token refresh; QuickBooks OAuth callback fix; Typesense OpenAPI spec; OpenAI / Exa / SerpAPI / Stripe UI Endpoint Test Reference docs; Google Forms enhancements.
Mar 16 – Mar 31, 2026	OpenFIGI (4 methods, API key auth); Notion UI test reference; GitHub `write_object_to_dataframe` refactor (list support); AWS RDS MySQL registry registration; Supabase (50 methods); Eventbrite; Google Meet; Apify (69 methods, webhook + wait-for-run).
Apr 1 – Apr 15, 2026	Attio; Instantly (63 methods); Chorus; NeverBounce; PostgreSQL parameter standardisation; LinkedIn Sales Navigator (with token refresh); MySQL connector list/DataFrame support;

Figure 4.2 shows how connectors appear in the FlowGenX application gallery, and Figure 4.3 shows the connected-apps page where end users manage their live authentications. The full categorical breakdown of the integration catalogue is given in Section 2.8 and need not be repeated here.

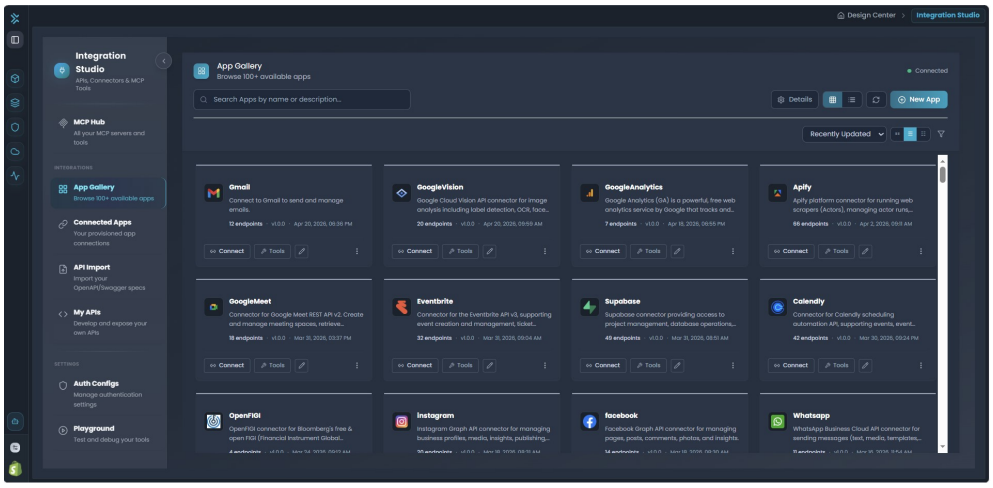

Figure 4.2: The FlowGenX AI app gallery, populated by connectors delivered through `runtime_connectivity`.

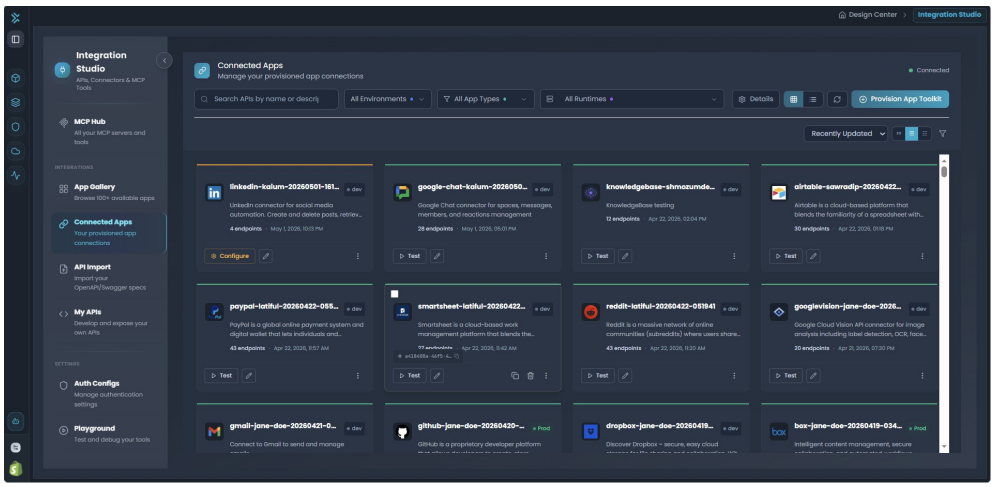

Figure 4.3: End-user view of connected apps (active authentications) on the FlowGenX platform.

4.4 Bug Fixes and Reliability Work

Alongside new feature delivery I shipped a number of reliability improvements. The most consequential were the **OAuth2 redirect-URI fixes** for QuickBooks and FreshBooks — the QuickBooks fix shipped together with an *access-token refresh mechanism* so that long-running sessions stopped failing with 401 errors — the **SQL Server retry logic** added to stabilise flaky connection tests under transient network errors (plus a firewall-rule mode-selection enhancement for Azure SQL Server), and the **Windows console encoding fix** in the Elasticsearch connector that prevented a Unicode crash on the Windows test runner and unblocked the Windows-based developers on the team. Smaller but production-relevant

fixes included WordPress connector hardening (`list_blocks` error handling, malformed base-URL construction in `search_content`), the Customer.io test configuration fix for an outdated API key, the Google Chat post-method malformed-request-body fix, a cache-cleanup fix that prevented stale data between connector test runs, Datadog log-ingestion debugging, AWS RDS MySQL registry registration, and GitLab revert coordination after a teammate's revert.

4.5 Code Improvement and Refactoring

Reliability and feature work were complemented by a handful of larger refactors. The most impactful was the **DataFrame / list duality across SQL connectors** — standardising parameter naming and accepted shapes across PostgreSQL, MySQL, and GitHub so that write methods transparently accept either a `pandas.DataFrame` or a plain `list[dict]`, which made the connectors significantly more flexible for downstream consumers that did not always have pandas in scope. The **AWS Textract** and **Anthropic** connectors were both restructured to separate sync and async document-analysis methods and to cleanly split the Files API from the new Skills API respectively. Smaller refactors included the `Zoho_CRM` folder rename to align with project casing conventions, an improved LaunchDarkly test script with cleanup logic and richer logging, and general readability passes on the FreshBooks endpoints (clients, expenses, invoices, tasks, time entries) and other resource-heavy connectors. Finally, the **OpenAPI sample backfill** replaced placeholder values in `request_schema` and `response_sample` with real values captured from live API runs (FreshDesk, Freshservice, GitLab, ClickUp, Apify, Instantly, Mailchimp, Freshsales, Freshworks, and others) — the work that directly improves the end-user experience by pre-filling forms with values that actually work.

4.6 Documentation Contributions

A connector at FlowGenX is not considered fully complete until it has a UI Endpoint Test Reference document — a concrete, hands-on cookbook that shows end users how to call each method from the platform UI, what parameters to provide, and what shape of response to expect. I authored such documents for many of the high-value connectors I delivered, including Notion, Stripe, OpenAI, Exa, SerpAPI, HelpScout, Typesense, Zoho CRM, Zoho Analytics, Zoho Campaigns, FreshDesk, Freshservice, AWS Rekognition, and Google Forms. These documents serve a dual purpose: they give end users immediate clarity on how to use a connector, and they give other engineers a ready-made test plan when they need to verify that a connector still works after an upstream API change.

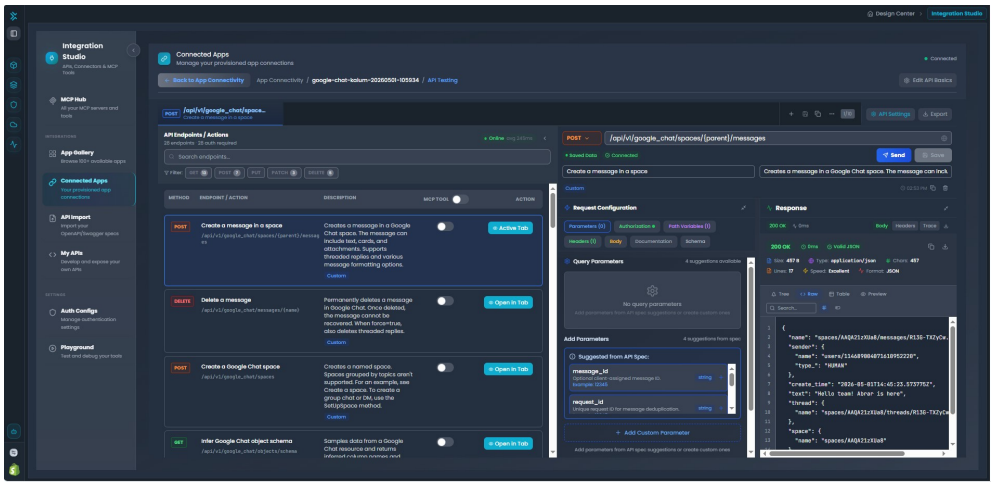

Figure 4.4: Per-connector endpoint testing UI used during validation.

4.7 Collaboration and Workflow Discipline

Across the entire engagement, the way I worked with the rest of the team mattered as much as the code I shipped. Each connector was developed on its own feature branch (`wordpress-connector`, `Harness_connector`, `MySQL-connector`, `Google-Forms-connector`, and so on); `dev` was the integration branch and was never committed to directly. Every commit followed the convention `feature/Rakib/(<connector>): <summary>`, so reviewers could trace work per integration without reading diffs. Feature branches were kept up-to-date with `origin/dev`, with non-trivial manual merges on the AWS Textract, HelpScout, X (Twitter), Harness, Google Forms, FreshBooks, Freshworks, and Zoho CRM branches. Every merge was paired with a re-run of the metadata pipeline (`generate_openapi_spec.py`, `generate_auth_config.py`, `populate_connector_db.py`) so the connector appeared in the FlowGenX UI immediately rather than requiring a follow-up pass.

The internship also gave me room to explore technologies beyond the strict scope of my connector work — the *Model Context Protocol (MCP)* (exercised through the MCP Playground for almost every connector I shipped), *Agent-to-Agent (A2A)* communication, gRPC patterns inside `runtime_apis`, the Apache Airflow side of the workflow engine, and the reusable `runtime_components` library (`FilterComponent`, `DataMappingComponent`, `DataPrivacyComponent` with PII masking, `JoinerComponent`, `HTTPNodeComponent`, `ForeachComponent` `SwitchComponent`, and the `SensorComponent`). These exposures, although not part of my shipped pull requests, gave me an end-to-end picture of how an agentic platform fits together.

4.8 Impact in Summary

Delivered 90+ production-ready connectors and standardised heavy APIs behind Manager classes, improving reliability (retry logic, token refresh, encoding and error-handling) and enabling broad platform operations. I also added real API samples, OpenAPI specs, and UI test-reference docs to speed adoption and make connectors easy for teammates to reuse.

Chapter 5

Skills Acquired

This chapter catalogues the skills I acquired during my internship at FlowGenX AI. They fall into two broad categories: technical skills, which I developed through the day-to-day work on the `runtime_connectivity` project and the surrounding repositories, and non-technical skills, which I developed through working inside a remote-first, globally distributed engineering team.

5.1 Technical Skills

5.1.1 Programming Languages

Python 3.11+ was the primary language of the internship and the one I now use most fluently. Across five and a half months and 178 commits I wrote idiomatic, typed, async-aware Python: Pydantic models, decorators, context managers, generators, dataclasses, asyncio coroutines, pytest fixtures, and mocking patterns. Black, isort, Ruff, and pre-commit hooks made the "correct shape" of clean Python second nature. **SQL** got daily exposure across PostgreSQL, MySQL, SQL Server, and the cloud managed-database variants (AWS RDS, Azure SQL): CRUD, schema inspection, bulk-write logic, and the parameter standardisation work that unified how SQL connectors accept both DataFrames and raw lists of records. **TypeScript / JavaScript** stayed at read-level: while my own contributions were primarily backend, I regularly read the Next.js frontend to understand how the UI consumed the auth-config and OpenAPI metadata my connectors produced. **Configuration languages** (JSON, YAML, TOML) were used pervasively for OpenAPI specs, Docker Compose, Kubernetes manifests, and Python project files.

5.1.2 Frameworks and Libraries

FastAPI and **Pydantic** sit at the centre of the platform's backend; I worked with FastAPI patterns through the runtime APIs and used Pydantic extensively to model connection parameters and request/response schemas for every connector. **Apache Airflow** is the workflow engine underneath `runtime_apis`, and I learned how DAGs are defined, scheduled, and triggered, and how the connectors I shipped become callable nodes inside them. **Pandas** drove the in-memory DataFrame transformations in the database and file connectors, and the DataFrame/list duality I introduced into the SQL write methods. **boto3** was used pervasively across the AWS connectors (S3, RDS, Textract, Rekognition) including paginators, signed-request flows, multipart uploads, asynchronous job patterns, and IAM credential handling. Other libraries used routinely include `requests`, `httpx`, `asyncio`, `pytest` (with fixtures, mocks, and integration markers), and `Jinja2`.

5.1.3 Databases

Hands-on familiarity with a wide range of database backends through the connectors I built and tested:

- **Relational:** PostgreSQL, MySQL, Microsoft SQL Server, Azure SQL Server, AWS RDS PostgreSQL, AWS RDS MySQL, Snowflake.
- **Document and NoSQL:** MongoDB.
- **Search:** Elasticsearch, Typesense.
- **Vector databases:** Pinecone, Weaviate, Milvus.

For each I learned the connection model, the authentication options, the standard CRUD shape, the pagination model, and the performance-relevant operations (bulk insert, upsert, truncate).

5.1.4 Cloud Platforms

On **AWS** I worked directly with S3 (bucket policies, versioning, multipart uploads), RDS for both PostgreSQL and MySQL, Textract (sync and async document analysis with adapters), Rekognition (image, video, face, custom labels, stream processors), and IAM credential flows. On **Azure** I worked with Blob Storage, Data Lake, and Azure SQL Server (with firewall-rule mode selection and retry logic). At the connector level I also built work on **Google Cloud** services (Tasks, Forms, Chat, Meet) and **Microsoft 365** (OneDrive and SharePoint).

5.1.5 AI / LLM Providers and Generative-AI Patterns

I built and tested provider connectors for **OpenAI** (assistants, models, threads), **Anthropic** (messages, batch, files, models, the Skills API), **Google Gemini**, **xAI**, **Groq**, **DeepSeek**, and **Perplexity AI**. The **vector-database** connectors (Pinecone, Weaviate, Milvus) gave me working familiarity with embeddings, similarity search, metadata filtering, and the role these systems play inside RAG pipelines. Working inside FlowGenX also exposed me to the practical use of multi-agent patterns (ReAct, Supervisor, Swarm, Deep) and the underlying protocols — the *Model Context Protocol (MCP)* for exposing tools to agents and *Agent-to-Agent (A2A)* communication for inter-agent collaboration.

5.1.6 Authentication Patterns

The connector work exposed me to every modern authentication pattern in production use:

- **OAuth 2.0** with refresh-token flows (the most important being the QuickBooks access-token refresh mechanism I implemented).
- **API Key** and **Bearer Token** authentication.
- **HTTP Basic Authentication**.
- **AWS IAM** credentials and signed-request flows.

5.1.7 System Architecture and Concepts

FlowGenX is composed of multiple cooperating services (`flowgenx-nextjs-V2`, `runtime_apis`, `runtime_components`, `runtime_connectivity`), and working inside this **microservice architecture** gave me a concrete intuition for service boundaries, package-level interfaces, and the discipline required to evolve a shared library without breaking its consumers. Every connector method I wrote produced **OpenAPI 3.0** metadata via the `@openapi_endpoint` decorator, so by the end of the internship I could read and write OpenAPI schemas fluently. I also implemented **webhook** configuration for connectors such as Apify and participated in the broader event-driven model of the runtime, including the *Webhook Trigger* that initiates workflow execution when an external system posts data. **gRPC** stayed at read-level through the runtime APIs codebase, and I worked with **Docker** for local development, observed the **Kubernetes / Helm / Terraform** setup the platform uses in production, and was directly exposed to caching patterns through the connector cleanup work and the SQL Server retry logic.

Beyond the connector layer, I gained a working understanding of several **platform-level products** that surround it: *Agent Fabric* (multi-agent orchestration runtime), *Agent Gateway* (governance/security plane), *VectorVerse* (vector-powered Knowledge Base), the *Playground* (interactive testing surface for MCP, agent, and workflow modes), the *API Workspace* (unified management for OpenAPI tools and authentication), and the *Data Compiler* (schema-drift component that keeps long-running pipelines resilient to upstream change) — giving me an end-to-end mental model of how an agentic platform fits together.

5.1.8 Testing

I wrote hundreds of **pytest** cases across the connectors I delivered (fixtures, mocks, parameterised tests, integration markers), plus a dedicated **live-API integration test script** for every connector that produced a PASS/FAIL/SKIP report against the real third-party API. Every connector was finally validated through the platform's API tester, MCP Playground, and end-to-end workflow execution.

5.1.9 Tooling and IDEs

- **Git and GitHub** — feature-branch workflow, pull requests, code review, merge conflict resolution.
- **ClickUp** — task tracking, status reporting, and sprint visibility.
- **uv** — the modern, fast Python package manager that FlowGenX standardises on.
- **Black, isort, Ruff, pre-commit** — the code-quality toolchain that enforces uniform style across the codebase.
- **Docker and Docker Desktop** — local container development.
- **Visual Studio Code** — my primary IDE for the internship, with Python, Pylance, and Docker extensions.
- **Postman** — exploration of third-party APIs before coding the connector.
- **Google Chat and Google Meet** — the daily communication tools.

5.2 Non-Technical and Professional Skills

5.2.1 Communication and Code Review

Working as the only intern on a remote, distributed team taught me to write more carefully: self-contained status updates, asynchronous phrasing of technical questions, blockers raised early in writing rather than in synchronous meetings, and questions asked in public channels (so answers benefited the team and stayed searchable). On **code review**, every pull request I opened received review comments, and I came to appreciate review as one of the highest-value activities a team can do — treating each comment as a conversation rather than a verdict, pushing back politely with reasoning when I disagreed, and thanking reviewers for catching subtle issues. By the second half of the internship I was occasionally reviewing teammates' pull requests myself, a small but meaningful sign of having earned the team's trust.

5.2.2 Time Management, Self-Direction, and Cross-Time-Zone Work

A remote internship with flexible hours requires more self-discipline, not less. I built the habit of setting a daily intention — "ship one connector to a reviewable state, fix one bug, or close one documentation deliverable" — and stayed honest with myself about what counted as "done". The ClickUp board and daily pull-request flow made this discipline tangible. Working across the roughly twelve-hour Dhaka-to-San Francisco offset taught me to leave detailed end-of-day updates so the United States team could pick up where I left off, to compose questions that did not require immediate replies, and to schedule synchronous meetings during the evening overlap window without disrupting either side's focus time.

5.2.3 Professionalism, Curiosity, and Resilience

The defining cultural expectation at FlowGenX — as captured in my offer letter — was that interns demonstrate the same commitment and work ethic as full-time team members, and I tried to honour that throughout: every commit followed the convention, every pull request was self-reviewed before opening, every connector was tested before being declared complete, every deliverable documented for the next person who would touch it. The breadth of technologies on display (90+ third-party APIs, multiple cloud providers, several agentic patterns, MCP, A2A, Airflow) forced a kind of structured curiosity — reading API documentation efficiently, skimming a codebase for patterns rather than reading every line. And every connector exposed at least one quirky issue (a malformed base URL, an undocumented rate limit, an auth flow that worked on Linux but not Windows), which I learned to treat as a puzzle with a finite solution rather than a blocker, and to document so the next person hitting the same issue would not start from scratch. The OpenAPI sample-backfill work was the clearest example of patience and attention to detail: replacing placeholder values with real ones across hundreds of endpoints is unglamorous but disproportionately impactful for the end-user experience.

Chapter 6

Life at FlowGenX AI

This chapter reflects on what it was actually like to work at FlowGenX AI as an intern. Because my internship was fully remote, this is necessarily a different kind of chapter from the "life-at-the-office" narratives common in earlier internship reports. Remote-first work is now the dominant model in modern software companies, and the lived experience of being an intern at a globally distributed startup deserves to be described on its own terms rather than as a poor substitute for an in-office one.

6.1 A Remote-First, Globally Distributed Workplace

FlowGenX AI is headquartered in San Francisco, California, with engineers based in the United States and Bangladesh. There is no shared physical office in Bangladesh; the company runs as a remote-first organisation, with all employees and interns working from their own homes or chosen workspaces. The expectations and support structures the company puts in place are designed for that reality: the tools, communication norms, and cultural cues collectively make distributed work feel coherent rather than fragmented.

The environment in practice has the following defining traits:

- **Trust over presence.** Nobody monitors when an engineer is online; what matters is whether the work moves forward, the pull requests get reviewed, and the standup updates are posted. This puts the burden of self-direction on the engineer, but in exchange offers a kind of working life that is very hard to find in traditional offices.
- **Asynchronous-first communication.** Most coordination happens through Google Chat threads and ClickUp comments. Meetings are reserved for design discussions, code-walkthroughs, and onboarding sessions. The team has been thoughtful about not over-scheduling video calls.
- **Globally distributed culture.** Working alongside engineers and leadership in San Francisco from Dhaka is itself a professional experience worth highlighting. It teaches habits — written clarity, asynchronous handoffs, time-zone empathy — that translate directly into how senior software roles operate today.

6.2 My Workspace and Equipment

Per my offer letter, the internship was full-time, remote (Bangladesh), and I used my personal laptop for all work. I set up a dedicated workspace at home with:

- A reliable broadband connection with a backup mobile hotspot for resilience.
- Local installations of Python 3.11, Docker Desktop, the `uv` package manager, Visual Studio Code, Postman, Git, and the GitHub CLI.

- Local clones of the four FlowGenX repositories under a single parent directory so that editable installs cross-referenced each other cleanly.
- A local PostgreSQL instance for connector-metadata development and testing.

The remote-first model meant I owned the operational health of my own development environment. When something broke (Docker on Windows having a moment, a Python virtual environment getting out of sync, a PostgreSQL upgrade requiring a re-import), I learned to diagnose and fix it myself rather than escalating to an IT team. This is not a small skill — and it is one that classroom education rarely teaches.

6.3 Daily and Weekly Cadence

6.3.1 A Typical Working Day

My typical working day ran from approximately **9:00 a.m. to 5:00 p.m. Bangladesh Standard Time (UTC+6)**, with flexibility to shift hours when a synchronous meeting with the United States team required late-evening overlap. A typical day looked roughly like this:

- **Morning (9:00 – 11:00).** Catch up on overnight activity from the United States team — Google Chat messages, pull-request comments, ClickUp updates. Resolve straightforward review feedback before starting deep work.
- **Late morning to early afternoon (11:00 – 2:30).** Focused implementation work on whichever connector or fix was the day's primary deliverable.
- **Afternoon (2:30 – 5:00).** Testing, integration test runs, sample-backfill work, and any outstanding documentation.
- **Evening (occasional).** Synchronous meetings or code-walkthroughs with the United States team during their morning, when an overlap was needed.

6.3.2 Weekly Rhythm

The week alternated between focused individual work and lightweight synchronous touchpoints. Pull requests typically opened and merged through the week as connectors were ready, rather than being batched into a single end-of-week push, and this rolling cadence is what made the team's velocity feel sustainable rather than spiky.

6.4 Communication and Collaboration Style

6.4.1 Tools

- **Google Chat** for asynchronous messaging — both one-on-one and team channels.
- **Google Meet** for synchronous discussions, design reviews, and onboarding sessions.
- **ClickUp** for sprint and task tracking.
- **GitHub** for code review through pull-request comments.

The team migrated from *Microsoft Teams* to *Google Chat* during my tenure, and I participated in the migration first-hand — a small but real experience of how a distributed team rolls out infrastructure changes without disrupting ongoing work.

6.4.2 Norms

A few unwritten norms shaped how the team worked: questions go into public team channels (rather than direct messages) so the answers remain searchable; messages are written to be self-contained so the reader does not need additional context to act; help requests show what has already been tried and observed rather than just "it does not work"; and questions are phrased with timezone empathy so they can be answered overnight and come back actionable in the morning.

6.5 Onboarding Experience

The onboarding period was deliberately short and project-driven. There was no multi-week classroom-style training; instead I was pointed at the project's internal documentation, walked through the architecture by my supervisor, and encouraged to start building real connectors as my learning artefacts. By the end of the second week I had merged my first half-dozen pull requests and had a complete mental model of the codebase.

For me personally this approach worked very well. Building real features from week two meant I was investing in the same code I would still be working on six months later, and the depth of understanding that comes from shipping is qualitatively different from the kind that comes from reading.

6.6 Mentorship and Learning Culture

The single most positive aspect of life at FlowGenX AI was the mentorship. My supervisor reviewed every pull request thoroughly, explained the reasoning behind every suggestion, and never made me feel that a question was too small. My team lead's design feedback on larger refactors gave me a sense of how senior engineers think about evolving a codebase. My teammates were generous with their time when I needed help understanding a piece of the runtime that sat outside my immediate work.

Beyond direct mentorship, the company maintains an active engineering blog (`https://blog.flowgen` and developer documentation (`https://docs.flowgenx.ai/`) that engineers are encouraged to read and contribute to. The Generative AI space moves fast, and working at FlowGenX meant being constantly nudged toward what was new in the field — new agent patterns, new providers, new protocols.

6.7 Cross-Time-Zone Collaboration and Reflection

Working from Bangladesh on a team partly based in San Francisco — roughly a twelve-hour offset, with only a few precious overlap hours per day — required intentional handoffs. I

learned to leave detailed end-of-day updates so the United States team could pick up where I left off, to compose questions that did not require an immediate reply (so the answer could come back overnight and be actionable in the morning), and to schedule the small number of synchronous meetings during the evening overlap window without disrupting either side's focus time. By the end of the internship, this discipline felt entirely natural and is one of the most concretely transferable skills the experience gave me.

There is sometimes a quiet assumption in the academic conversation about internships that "real" internships happen in offices. My experience challenged that assumption. What I missed — the casual hallway conversations, the in-person whiteboard sessions — I missed honestly. But what I gained was substantial: the experience of contributing to a serious product alongside experienced engineers in San Francisco, California, with the agency and trust that comes from being treated as a contributing member of the team from day one.

Chapter 7

Conclusion

This internship at FlowGenX AI bridged the gap between seven semesters of classroom learning at the Department of Software Engineering, IICT, SUST, and professional software engineering practice. Over five and a half months, I contributed 178 commits and developed more than ninety production-ready connectors for `runtime_connectivity`.

The internship taught me three key lessons. First, software engineering at scale is the art of being consistent. The connector lifecycle — API research, ConnParams, Manager, decorated methods, integration tests, backfill, metadata generation, and documentation — worked because it remained consistent. Second, small details matter: proper field names, correct pagination handling, and pre-filled example values distinguish frustrating features from delightful ones. Third, mentorship multiplies growth. The patient code reviews and architectural guidance I received shaped my development significantly.

FlowGenX AI exemplifies the kind of company I aspire to work at: small enough for intern contributions to be visible, ambitious enough for the work to matter, and disciplined enough to maintain craft under pressure. The distributed team demonstrated how mature engineers across continents can collaborate effectively.

I am grateful to the Department of Software Engineering and IICT at SUST for organizing the internship programme; to **Mr. Sumon Saha**, Founder & CEO of FlowGenX AI, for his mentorship and opportunities; to my supervisor Mr. Abrar Bin Rofique for daily guidance; to Mr. Ashikuzzaman Abir for workflow-agent insights; to my team lead Mr. Sanjib Sen for architectural direction; and to my teammates Mr. Nayem Islam and Mr. Latiful Kabir for their collaboration.

I leave this internship more confident as a backend engineer, fluent in the Generative AI stack, and determined to build software that matters. The knowledge and habits gained here form the foundation for my career ahead.

References

The following sources have been consulted in the preparation of this report. URLs were last verified at the time of submission.

1. FlowGenX AI — Official Website.
   `https://www.flowgenx.ai/`
2. FlowGenX AI — About Page (mission, vision, capabilities).
   `https://www.flowgenx.ai/about`
3. FlowGenX AI — Platform Overview.
   `https://www.flowgenx.ai/platform/workflow-development`
   `https://www.flowgenx.ai/platform/work-ai`
   `https://www.flowgenx.ai/platform/integration-studio`
   `https://www.flowgenx.ai/platform/knowledgebase`
   `https://www.flowgenx.ai/platform/security-governance`
   `https://www.flowgenx.ai/platform/agent-fabric`
4. FlowGenX AI — Use Cases.
   `https://www.flowgenx.ai/usecases`
5. FlowGenX AI — Templates.
   `https://www.flowgenx.ai/templates`
6. FlowGenX AI — Frequently Asked Questions.
   `https://www.flowgenx.ai/FAQ`
7. FlowGenX AI — Developer Documentation.
   `https://docs.flowgenx.ai/`
8. FlowGenX AI — Engineering Blog.
   `https://blog.flowgenx.ai/`
9. FlowGenX AI — LinkedIn Page.
   `https://www.linkedin.com/company/flowgenx-ai/`
10. FlowGenX AI — Application Platform.
    `https://platform.flowgenx.ai/`
11. Internal documentation of the `runtime_connectivity`, `runtime_apis`, and `runtime_components` repositories of FlowGenX AI (referenced with the company's permission; specific contents omitted under confidentiality).
12. Author's personal Git commit history on the `runtime_connectivity` repository, period 12 November 2025 – 02 May 2026.
13. Author's internship offer letter, dated 22 October 2025, issued by FlowGenX AI.
14. Anthropic — Claude Developer Documentation.
    `https://docs.anthropic.com/`
15. OpenAI — API Reference.
    `https://platform.openai.com/docs/`
16. Model Context Protocol — Specification.
    `https://modelcontextprotocol.io/`